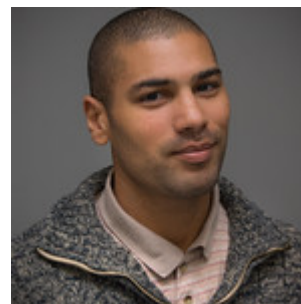


Scripting en sistemas operativos.

Caso práctico

En un avance en el conocimiento de los administradores de equipos, el departamento de informática afronta cómo formar a los componentes del departamento para capacitarles en la creación de guiones o scripts en los distintos equipos, y con distintos sistemas operativos que tenemos en la empresa.

Vindio y **Laro** han demostrado altos conocimientos en administración de servidores. Sobre todo **Vindio** en administración de servidores de libre distribución GNU/Linux, y **Laro** en servidores de la factoría Microsoft (Windows Server).



[Alain Bachellier](#) (CC BY-NC-SA)

Todos están de acuerdo en que son las personas más adecuadas para formar al departamento en el diseño, e implementación según necesidades de programas scripts o guiones a implantar y según las necesidades en equipos Windows o GNU/Linux.

Laro y **Vindio** deciden reunirse para recopilar toda la información disponible y seguir unas pautas para realizar las ponencias.

En esta unidad de trabajo aprenderás conceptos importantes, como integrar programas de scripting en sistemas operativos de distinta factoría.

Con esta unidad se pretende que conozcas los conceptos propios sobre integración de comandos y variables de entorno en su variante de programas de scripting en sistemas operativos tanto propietarios, por ejemplo Windows, como libres, por ejemplo GNU/Linux.

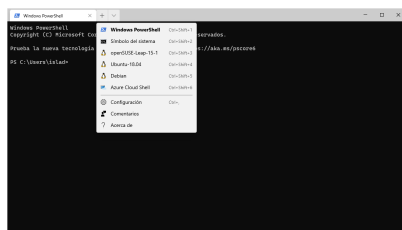


[Ministerio de Educación y Formación Profesional](#) (Dominio público)

**Materiales formativos de FP Online propiedad del Ministerio de
Educación y Formación Profesional.**

[Aviso Legal](#)

En el caso de distribuciones GNU/Linux accedemos a la terminal a través del menú o bien cambiando de terminal. ¿Cómo? Mediante la combinación de teclas `<ctrl><alt><Fn>` (sustituyendo `Fn` por la terminal a la que queramos acceder; por ejemplo: `F1` terminal 1, `F2` terminal 2, etc.).



Windows 10 (Elaboración propia)

Actualmente Microsoft trabaja con **Powershell**. Un sistema shell más potente con su propio lenguaje de programación y entorno gráfico de desarrollo.

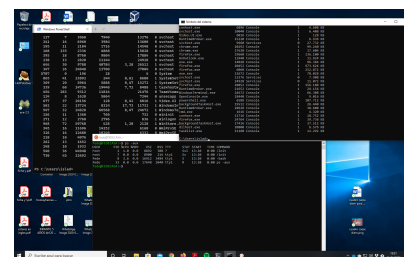
También tenemos que tener presente que Microsoft está potenciando mucho la integración del subsistema **WSL** dentro de Windows, tanto en servidores como en clientes; y una herramienta clave en ese sentido es **Windows Terminal**, que integra en pestañas los distintos shell que haya instalados en el sistema: símbolo del sistema, Powershell, bash de WSL, etc.

1.1.- Tipos de shell.

Si nos ceñimos a los sistemas operativos más conocidos hoy en día, podemos indicar los siguientes intérpretes shell.

GNU/Linux:

Tenemos diversos intérpretes (sólo los más utilizados en GNU/Linux): `sh` (Bourne shell), `bash` (Bourne-Again shell), `ksh` (Korn shell) y `zsh` (Z shell).



Windows 10 (Elaboración propia)

Para saber más

En Wikipedia podemos tener un punto de partida bastante interesante para conocer más los distintos shells que puede usar una distribución de GNU/Linux.

[Shells en cualquier distribución GNU/Linux.](#)

Windows:

Básicamente tenemos dos: `cmd` y `powershell`.

`Cmd` es el más conocido y puede interpretar scripts o guiones realizados en ficheros Batch (extensión BAT), o Visual Basic Scripts (extensión VBS).

Para saber más

En el siguiente enlace nos introduce en las sintaxis y estructuras de ficheros por lotes (.bat). Es un buen comienzo.

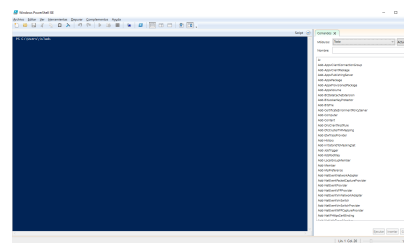
[Sintaxis y estructuras por lotes de ficheros batch.](#)

En el siguiente enlace tenemos todos los comandos que se pueden ejecutar utilizando la línea de comando de Windows.

[Comandos para ejecutar desde cmd.](#)

A día de hoy, **Powershell** está siendo adoptado como intérprete de comandos principal en todos los sistemas Windows. Mucho más potente y con herramienta de edición y depurador incluidos.

En la imagen de la derecha se muestra el editor que incorpora y que tiene como ventaja la depuración o prueba de un comando para su inserción en el script, Powershell ISE.



Windows 10 (Elaboración propia)

Para saber más

Partiendo de la Wikipedia, podemos obtener bastante información sobre Powershell

[Powershell en la Wikipedia](#)

Documentación oficial de Microsoft sobre Powershell

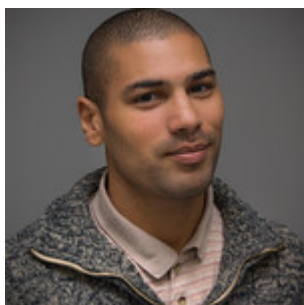
[Documentación oficial sobre Powershell 7](#)

En este otro enlace tenemos una guía muy interesante

[Guía de Powershell para principiantes](#)

2.- Estructuras del lenguaje.

Caso práctico



[Alain Bachellier](#) (CC BY-NC-SA)

Siguiendo con la dinámica de preparación de documentación por parte de **Laro** y **Vindio**, estos revisan la sintaxis y estructura de las distintas instrucciones de los lenguajes de scripts que se utilizarán en las distintas ponencias.

El intérprete de comandos o "shell" es también un lenguaje de programación completo. La programación de shell se usa mucho para realizar tareas repetidas con frecuencia y que no requieren programas complejos. Los administradores de sistemas suelen escribir programas o aplicaciones en el lenguaje de base del sistema operativo por razones de rapidez y eficiencia.

Como todo lenguaje, debe cumplir con unos requisitos tales como emplear la sintaxis correcta, que no tenga errores, no sólo sintácticos sino también lógicos.

Ese tipo de lenguaje de programación consta de órdenes. Órdenes que, generalmente, podremos usar directamente en un entorno tipo terminal. Y aparte de estas órdenes, disponemos de instrucciones con una estructura que nos permite jugar con las órdenes del sistema.

Pondremos un ejemplo de un script en GNU/Linux:

```
#!/bin/bash
# misdatos.sh
# muestra datos relativos al usuario que lo invoca
#
echo "MIS DATOS."
echo " Nombre: "$LOGNAME
echo "Directorio: "$HOME
echo -n "Fecha: "
date
echo
# fin misdatos.sh
```

Este script muestra datos que tiene el sistema acerca del usuario que ha ejecutado el script: su nombre de inicio de sesión (\$LOGNAME), su carpeta de inicio, y la fecha actual.

Para saber más

En el siguiente enlace podrás estudiar las estructuras del lenguaje de scripts para los distintos tipos de shell de GNU/Linux.

[Estructuras del lenguaje script para los distintos tipos de shell en GNU/Linux](#)

Otro recurso interesante que introduce en la estructura shell en Windows (cmd) y en GNU/Linux.

[Estructura Shell en Windows y en GNU/Linux](#)

En el siguiente enlace, hay acceso a un curso online de introducción a Powershell

[Curso de powershell para Windows](#)

Autoevaluación

¿Qué entendemos por "shell"?

- ☐ Es un intérprete de comandos utilizado por sistemas operativos.
- ☐ Es el entorno de navegación de cualquier sistema operativo.
- ☐ Es cualquier intérprete o compilador de guiones o scripts.
- ☐ Es un entorno de programación a nivel máquina.

Sí fenomenal, es cierto.

No es cierto, ¿Cómo has elegido esta respuesta? Repasa el tema.

Respuesta incorrecta, creo que deberías leer de nuevo el tema.

No, no es nada tan complejo.

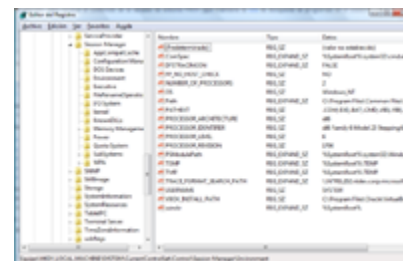
Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto
4. Incorrecto

2.1.- Variables de entorno.

No debemos olvidar que, para interactuar más eficientemente con el sistema operativo, debemos tener un sistema cómodo para comunicarnos más fiablemente y conseguir que esas rutinas que implementamos sean lo más eficientes posibles. Esto lo conseguimos, por ejemplo, con la utilización de las variables de entorno.

Sabemos que todo lenguaje de programación tiene distintas maneras de guardar información o datos a medida que se va generando. Los datos en memoria se almacenan en variables. Bien, estas variables pueden generarse en el propio programa o bien pueden pasarse como parámetros al programa o bien, en el caso de scripting, puede utilizar las variables de entorno que usa el sistema operativo.



Windows 10 (Elaboración propia)

¿Qué son las variables? Es el nombre interno que le damos a una porción de memoria donde almacenamos un dato.

Y, ¿una variable de entorno? Una variable de entorno es un objeto designado para contener información usada por una o más aplicaciones. Algunos usuarios encuentran esto un poco extraño o inmanejable. Sin embargo esto no es cierto: usando variables de entorno hace que cualquiera pueda cambiar una opción de configuración para una o más aplicaciones fácilmente. En Windows, podremos observar las variables de entorno por defecto en las claves de registro `HKEY_CURRENT_USER\Environment` y del sistema en `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Environment`. No obstante, igual que en GNU/Linux, podemos leer todas las variables de entorno de nuestro equipo ejecutando, mediante Powershell, el cmdlet `Get-ChildItem Env:`, o manejando directamente la variable `$env`.

Veamos unos ejemplos:

[En Windows, con Powershell](#) [En GNU/Linux](#)

En Windows, con Powershell

```

> Seleccionar Administrador: Windows PowerShell

PS C:\WINDOWS\system32> Get-ChildItem Env:\COMPUTERNAME
Name                                     Value
----                                     -
COMPUTERNAME                           FEDECASA

PS C:\WINDOWS\system32> echo $env:COMPUTERNAME
FEDECASA
PS C:\WINDOWS\system32>

```

Windows 10 (Elaboración propia)

También podemos crear variables de entorno y modificar su resultado.

```

> $env:TestVariable = "Test variable"
PS C:\WINDOWS\system32> Get-ChildItem Env:\TestVariable
Name                                     Value
----                                     -
TestVariable                           Test variable

PS C:\WINDOWS\system32>

```

Windows 10 (Elaboración propia)

En GNU/Linux

Si ejecutamos `set` obtendremos todas las variables de entorno del sistema.

```

federico@FEDECASA:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/mnt/c/Program Files/WindowsApp
s/CanonicalGroupLimited.Ubuntu18.04onWindows.2020.1804.7.0.x64_79rhkplfndgsc:/mnt/c/Program Files (x86)/Common Files/O
racle/Java/javapath:/mnt/c/Program Files (x86)/Intel/Intel(R) Management Engine Components/ICLS:/mnt/c/Program Files/Int
el/Intel(R) Management Engine Components/ICLS:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/
c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files (x86)/Intel/Intel(R) Ma
nagement Engine Components/DAL:/mnt/c/Program Files/Intel/Intel(R) Management Engine Components/DAL:/mnt/c/Program Files
(x86)/Intel/Intel(R) Management Engine Components/IPT:/mnt/c/Program Files/Intel/Intel(R) Management Engine Components/
IPT:/mnt/c/Program Files/PuTTY:/mnt/c/Program Files/Calibre2:/mnt/c/Program Files/AutoFirma/AutoFirma:/mnt/c/Program F
iles/Microsoft VS Code/bin:/mnt/c/Users/islad/AppData/Local/Microsoft/WindowsApps/snap/bin
federico@FEDECASA:~$ export PATH=$PATH:/home/usuario/bin
federico@FEDECASA:~$

```

Federico Martínez Pérez (Elaboración propia)

Debes conocer

Manejo de variables en los sistemas Windows y GNU/Linux.

[Variables de entorno en Windows 10](#)

[Capítulos sobre Variables en PowerShell - Guía de Powershell para principiantes](#)

[Variables de entorno en GNU/Linux](#)

[Trabajando con las variables en GNU/Linux](#)

Autoevaluación

¿Los scripts pueden utilizar y manipular las variables de entorno?

- ☐ Sí, pero deben asignársela a una variable propia del programa.
- ☐ No, sólo se utilizan como información al usuario.
- ☐ Sí, si lo pasamos como parámetro al programa de script.
- ☐ Sí, se pueden utilizar y manipular.

No es cierto, las variables de entorno no dejan de ser variables.

Respuesta incorrecta, los scripts actúan como instrucciones del sistema operativo, no lo olvidemos.

No, las variables de entorno actúan como variables globales.

Muy bien, has entendido el concepto.

Solución

1. Incorrecto

- 2. Incorrecto
- 3. Incorrecto
- 4. Opción correcta

Autoevaluación

¿En GNU/Linux podemos usar variables de entorno?

- ☐ Sí.
- ☐ Sí, pero solo el usuario `root`.
- ☐ No, solo en sistemas operativos de Microsoft se pueden utilizar este tipo de variables.
- ☐ No, ni en Windows ni en GNU/Linux.

Muy bien, en todos los sistemas operativos hay variables de entorno.

No, todos los usuarios pueden usar las variables de entorno.

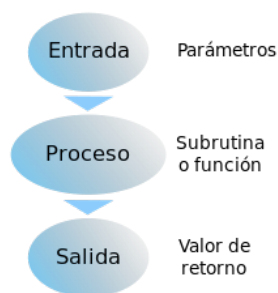
No es correcto, estás confundido.

Respuesta incorrecta. Deberías repasar los contenidos.

Solución

- 1. Opción correcta
- 2. Incorrecto
- 3. Incorrecto
- 4. Incorrecto

2.2.- Paso de parámetros.



Hemos visto cómo trabajar con las variables del sistema, pero no es la única forma que tenemos de intercambiar datos con nuestros scripts. En muchas ocasiones, sabemos que los comandos del sistema reciben parámetros que modifican su ejecución: bien ficheros desde los que reciben datos, ficheros para almacenar resultados, u opciones que modifican la ejecución del comando. Esto, por supuesto, se aplica también a los scripts que nosotros mismos creamos.

[Josell7](#) (CC BY-SA)

Debes conocer

Puedes aprender todo acerca de los parámetros en scripts de Bash en el siguiente enlace:

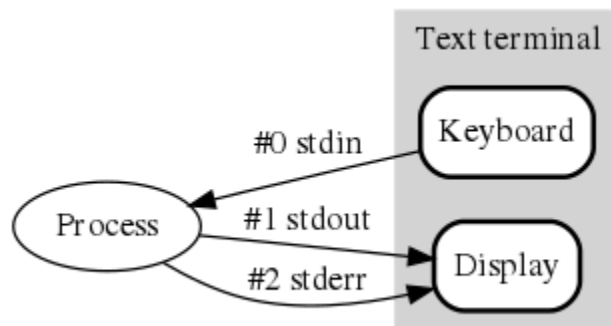
[Parámetros y opciones en el scripting de Bash](#)

En este otro enlace, tienes una introducción resumida a Powershell y al uso de parámetros en sus scripts:

[Scripting en Powershell](#)

2.3.- Redirección, tuberías y filtros.

Aunque no suele usarse dentro de los scripts, sino para complementar su funcionamiento, la redirección y las llamadas tuberías o "pipelining", así como los filtros de formato y contenido son tremendamente necesarios para trabajar con la shell.



[Danielpr85](#) (Dominio público)

- ✓ Redirección: Normalmente, se utilizan como flujos de entrada y salida estándares el teclado (entrada) y la ventana de shell (salida). Incluso, los mensajes de error también tienen un flujo estándar (que es la misma salida estándar).
- ✓ Filtros: es muy importante buscar, ordenar, organizar... la salida de datos. Mediante filtros, podemos ordenar dicha salida, mostrar parte de ella (primeras líneas, buscar determinadas columnas), realizar búsquedas...
- ✓ Tuberías: Mediante este mecanismo, enviamos la salida de una aplicación como entrada de otra. Esto sirve, por ejemplo, para usar los filtros.

Cuando hablamos de filtros y Bash, es importante hablar también de comandos como `awk` y `sed`.

- ✓ El primero, `awk`, es una herramienta muy poderosa: tiene todo un lenguaje de programación para procesar texto. Suele usarse para extraer información de la salida de datos, pero sirve para procesar datos de formas muy diversas.
- ✓ El segundo, `sed`, es un editor de texto no interactivo. Es perfecto para modificar ficheros de configuración sobre la marcha, por ejemplo, o buscar y reemplazar caracteres sobre un fichero.

Debes conocer

En los siguientes enlaces, tenemos información abundante sobre lo que hemos visto en este punto sobre Bash.

[Filtros, redirecciones y tuberías](#)

[Uso de comando awk](#)

[Uso del comando sed](#)

En estos recursos, vemos cómo se hace todo esto en Powershell.

[Redirecciones y tuberías en Powershell](#)

[Canalización en Powershell](#)

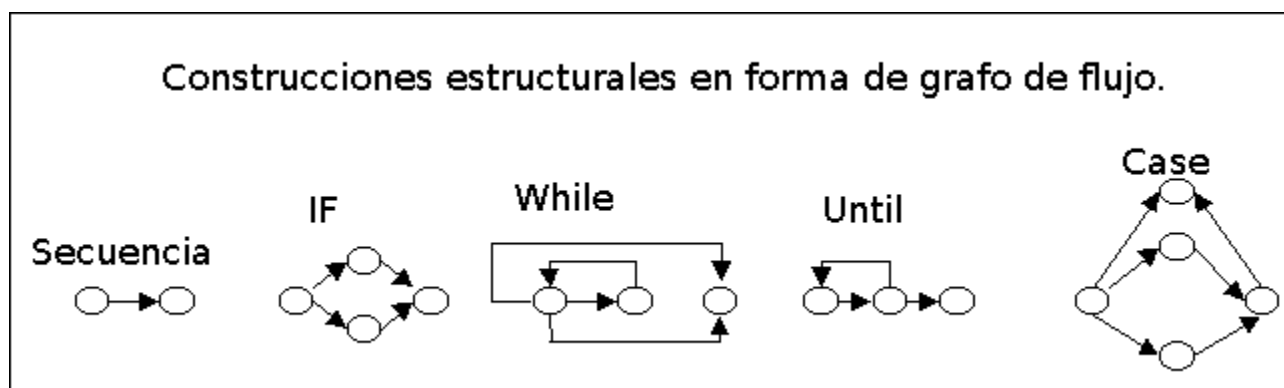
Y en este otro, todo lo necesario sobre el formato de salida y los filtros.

[Formato de salida en Powershell](#)

2.4.- Control de flujo.

Otro apartado fundamental a la hora de trabajar con un lenguaje de programación, es el control de flujo: bucles, condicionales, etc.

- ✓ Bucles y estructuras repetitivas: Nos permiten repetir un número determinado de veces operaciones: recorrer matrices o ficheros, realizar funciones matemáticas iterativas...
- ✓ Estructuras condicionales: nos permiten establecer condiciones a la hora de ejecutar partes del código: hacer una cosa u otra según una operación resulte, por ejemplo, positiva o negativa, etc.



[Waeswaes \(CC BY-SA\)](#)

Debes conocer

En el siguiente enlace tenemos información sobre estructuras iterativas o repetitivas en bash

[Estructuras Iterativas o Repetitivas en bash](#)

Y en este otro, sobre estructuras condicionales:

[Estructuras condicionales en Bash](#)

Y en este otro, información sobre control de flujo en Powershell:

[Capítulo Estructuras condicionales - Guía para principiantes de Powershell](#)

[Capítulo Estructuras repetitivas - Guía para principiantes de Powershell](#)

[Control de flujo en Powershell - Documentación de Microsoft](#)

3.- Creación y depuración de «scripts».

Caso práctico



[Alain Bachellier](#) (CC BY-NC-SA)

Uno de los inconvenientes con los que se encontrarán los desarrolladores de scripts o guiones, comentan **Vindio** y **Laro**, será saber cómo pueden depurar los distintos lenguajes de scripts que deberán utilizar.

Deciden, de común acuerdo, revisar su documentación e intentar actualizarla mediante visitas virtuales a los desarrolladores de las herramientas a través de Internet.

¿Cómo crearemos un programa de scripting en GNU/Linux o en Windows? Bien, para realizar un programa interpretado no necesita grandes herramientas de diseño. En Windows podremos utilizar el bloc de notas (`notepad`) y en GNU/Linux podemos usar herramientas de procesador de textos como `nano`, `vi` o herramientas gráficas como `gedit`, aunque por supuesto es posible usar algún otro editor con elementos de ayuda adicionales.

En el caso de Windows los archivos de script de la antigua línea de comando utilizarán una extensión, `.bat`, o `.cmd`. Es decir, un programa que llamamos "direccionar" lo guardaríamos como "direccionar.bat". Simplemente con colocar la extensión, el programa adquiere la categoría de ejecutable.

Con Powershell, utilizaremos su propio entorno de desarrollo, Powershell ISE, una herramienta integrada muy completa. Los scripts se almacenan con la extensión `.ps1`. Para su ejecución, de forma similar a GNU/Linux, tenemos que usar el nombre del shell delante; por ejemplo, un script llamado `direccionar.ps1` tendría que invocarse para su ejecución de la siguiente manera: `powershell -f direccionar.ps1`. También es importante recordar en Powershell que, por defecto, la ejecución de nuestros scripts está desactivada por motivos de seguridad: los scripts son una forma muy directa y potente de acceder a los recursos del sistema, y es importante saber qué se está haciendo en cada momento.

Para activar la ejecución de nuestros scripts locales, basta con ejecutar el siguiente comando:

```
Set-ExecutionPolicy RemoteSigned
```

En el caso de GNU/Linux no requiere extensión pero suele utilizarse una extensión más bien por reconocimiento a simple vista más que como imprescindible. La extensión que suele utilizarse es ".sh". Siguiendo con el ejemplo anterior sería "direccionar.sh". En GNU/Linux, las extensiones no tienen la importancia que tienen en Windows; el script podría tener otra extensión, o ninguna, y seguiría siendo igual de válido.

Para ejecutar un programa en el shell de GNU/Linux debemos realizarlo con la orden `sh`. Por ejemplo: `sh direccionar.sh`.

Podemos, también, cambiarle las propiedades al programa y le añadimos la propiedad de ejecutable al programa con la orden `chmod`. En ese caso, podríamos ejecutar directamente el fichero, indicando su ubicación dentro del árbol de directorios. Lo más sencillo es situarnos en la misma carpeta donde esté almacenado y ejecutarlo desde ahí: `./direccionar.sh`.

En GNU/Linux, al existir diversas `shell`, es conveniente utilizar la primera línea para indicar la shell que utilizará el programa.

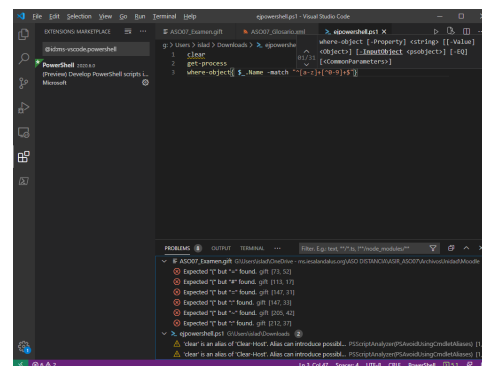
Como ejemplo: `#!/bin/bash` utilizará la shell `bash`.

Cuando tenemos un shell script o fichero por lotes grande y observamos, en su ejecución, que está fallando, normalmente empleamos el comando "`echo`" para imprimir una traza y ver por donde se queda el script.

Sin embargo, para depurar el código, el comando `sh` que utilizamos para "lanzar" un guión tiene un parámetro que nos permite visualizar los movimientos del guión, el valor de sus variables, etc.

Es importante disponer de una herramienta que podamos utilizarlo de "debugger" y seguir, paso a paso, la ejecución del script. De esta manera, podremos comprobar el código de funciones y/o subrutinas inspeccionando el valor de cada una de las variables.

Hay muchas herramientas de este tipo, pero estos últimos años ha destacado una en particular: VisualStudio Code. Admite montones de extensiones que le permiten trabajar con múltiples lenguajes, tanto como editor simple como en modo de depuración. Como gran ventaja, nos permite trabajar con scripts tanto de Bash u otras shell de GNU/Linux, como con Powershell, y funciona en sistemas Windows y GNU/Linux exactamente igual.



Windows 10 (Elaboración propia)

Debes conocer

[Depuración en GNU/Linux](#)

[Depuración en Windows](#)

Depuración en GNU/Linux

Depuración de un programa es el proceso de identificar y corregir los errores de programación que se generan en su construcción.

En inglés se le conoce como debugging, ya que se asemeja a la eliminación de bichos (bugs), manera en que se conoce informalmente a los errores de programación.

Se dice que el término bug proviene de la época de las computadoras de válvulas de vacío, en las cuales los problemas se generaban por los insectos que eran atraídos por las luces y estropeaban el equipo.

Depurar el código sirve para ver como se ejecuta paso por paso el script o programa, qué valores toman sus variables, si se ha cometido un fallo y saber en qué parte del código ha ocurrido, etc. No es algo que se deba hacer obligatoriamente con cada script que se programe, pero facilita mucho a la hora de buscar los problemas que surgen a la hora de programar.

Hay dos formas:

- ✓ Bien en la línea `#!/bin/bash -x | -v`
 - ➡ `-x` → Muestra las instrucciones antes de ejecutarlas por consola, y dando opción a sustituir las variables
 - ➡ `-v` → Muestra todo pero sin sustituir las variables.
- ✓ Mediante set cambiando las opciones de ejecución de la shell a lo largo del script, también con las opciones `-x | -v`.

Por ejemplo:

```
# Este script visualiza quien eres y el directorio en el que se encuen
#!/bin/bash -x
echo "Fecha y hora: `date`"
echo
echo "Tu usuario es: `whoami` \n"
echo "Tu directorio actual es: `c`"
pwd
```

o

```
sh -x horayquiensoy.sh argumento1 argumento2 argumentoN
```

Del mismo modo, volvemos a comentar que Visual Studio Code incluye herramientas para programar y depurar shell scripts en GNU/Linux.

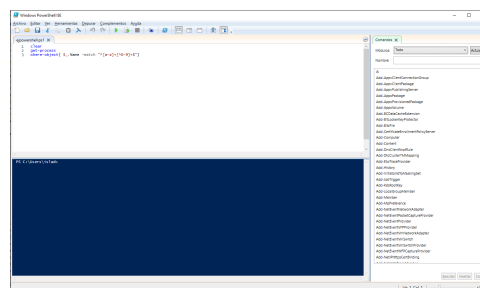
Depuración en Windows

En archivos bat o cmd (Windows) no existe un depurador como tal. Sí que podemos utilizar nuestro ingenio para generar mensajes durante la ejecución para comprobar el buen funcionamiento del programa de script.

Sin embargo, con Powershell sí podemos utilizar un depurador.

Podemos utilizar su propio entorno de edición como sistema de ventanas y depuración, Powershell ISE.

[Guía para principiantes de Powershell - el entorno de desarrollo ISE](#)



Windows 10 (Elaboración propia)

En el siguiente enlace de Microsoft podemos estudiar cómo crear puntos de interrupción y demás procesos de depuración y poder realizar un programa correcto sintácticamente y lógicamente en Powershell.

[Depuracion de scripts Powershell](#)

En este otro enlace, vemos un tutorial algo más básico.

[Tutorial sobre PowerShell ISE](#)

Para saber más

En el siguiente enlace, vemos más detalles acerca de la habilitación/deshabilitación de scripts en Windows:

[Habilitar la ejecución de scripts en Windows](#)

Como ya dijimos antes, Visual Studio Code también lleva herramientas para trabajar fácilmente con Powershell. En el siguiente enlace, explica cómo.

[Documentación de Microsoft sobre el uso de Visual Studio Code para desarrollar scripts de Powershell](#)

Autoevaluación

Para ejecutar un script en GNU/Linux desde un terminal debe...

- ☐ Tener la extensión sh.
- ☐ Es suficiente que contenga la primera línea `#!/bin/bash`.
- ☐ Con la instrucción `sh script` o cambiando los derechos del script.
- ☐ Con la instrucción `sh script` y cambiando los derechos del script.

No es correcto, en Linux no existen extensiones de archivos.

Respuesta incorrecta, algo más, falta algo más.

Muy bien, has entendido el planteamiento.

No, no son necesarios ambos requisitos.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

Autoevaluación

Sólo Windows necesita una extensión para identificar los scripts.
¿Verdadero o falso?

- ☐ Verdadero.
- ☐ Falso.

Muy bien, has visto que hay diferencias entre Windows y GNU/Linux.

No es correcto, ¿GNU/Linux desde cuando necesita una extensión de archivos?

Solución

1. Opción correcta
2. Incorrecto

Autoevaluación

¿Cuándo se puede utilizar, en GNU/Linux, extensiones de archivo?

- ☐ Nunca.
- ☐ No son necesarias, pero el usuario puede utilizarlas como referencia.
- ☐ Solo si son scripts o guiones, por ejemplo documentos html.

No es correcta, ¿El usuario no tiene criterio?

Muy bien, lo has entendido.

No, respuesta incorrecta, deberías repasar el apartado anterior.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

4.- Particularidades de Windows.

Caso práctico

A diferencia de GNU/Linux, Windows mantiene una base de datos llamada registro, comenta **Laro**. Con lo cual, **Vindio** se salva de esta parte de ponencia.

Vindio comenta, sonriendo, que es una de las causas por las que prefiere GNU/Linux.

Del mismo modo, **Vindio** le comenta a **Laro** si tiene en cuenta que Windows tiene un sistema de extensiones de comandos que permite interactuar de forma más completa con el sistema operativo.

A lo que responde **Laro** que ya lo tenía valorado. Contará cómo afecta al sistema, dónde encontrarlo en el registro y como habilitarlo a través de scripts.



[Alain Bachellier](#) (CC BY-NC-SA)

Los sistemas operativos Windows tienen diferencias estructurales con respecto a GNU/Linux. Una de las más importantes es el uso de una base de datos ordenada por tablas Hash, el registro, que almacena toda la información del sistema; GNU/Linux no tiene nada parecido.

A nivel de programación de scripts, también hay algunas diferencias:

- ✓ Para la línea de comando, existen las llamadas extensiones de comandos, que amplían la funcionalidad de las órdenes utilizadas en la shell y los scripts.
- ✓ Powershell permite la ampliación del lenguaje utilizando los llamados módulos, a modo de librerías del lenguaje, que aportan nuevos cmdlets, funcionalidades y objetos.

Veamos algunas de estas particularidades más a fondo.

4.1.- Extensiones de comandos específicos.

¿Qué son extensiones de comandos? Las extensiones de comandos se utilizan para ampliar la funcionalidad de determinados comandos del sistema y sólo en sistemas operativos Windows, para la interfaz de línea de comando. No se aplica a Powershell. Los comandos a los que afectan los puedes ver con `cmd /?` pero son los típicos de scripting: control de archivos (`del`, `cd`, `md`,...), control de rutinas (`for`, `if`, `goto`,...), etc...

¿Cómo sabemos que está habilitado? Si revisamos el registro y accedemos a la variable de registro `HKEY_CURRENT_USER\Software\Microsoft\Command Processor\EnableExtensions` `O` `HKEY_LOCAL_MACHINE\Software\Microsoft\Command Processor\EnableExtensions`.

¿Quieres un ejemplo? Utilizamos el comando `SET` para crear nuevas variables de entorno y probamos si podemos aplicar lo asignado.

```
C:\>SET micasa=C:\Documents and Settings\sirio
C:\>CD %micasa%
C:\Documents and Settings\sirio>
Z:\>SET m=ping -n 1 www.madrid.org
Z:\>%m%
Haciendo ping a www.madrid.org [195.77.128.9] con 32 bytes de datos:

Respuesta desde 195.77.128.9: bytes=32 tiempo=46ms TTL=247

Estadísticas de ping para 195.77.128.9:
Paquetes: enviados = 1, recibidos = 1, perdidos = 0 (0% perdidos),
Tiempos aproximados de recorrido redondo en milisegundos:
mínimo = 46ms, máximo = 46ms, promedio = 46ms
```

Vamos a indicar como habilitar las extensiones de comandos si éstas no están habilitadas. Esto se realiza con el flag o bandera `enableextensions`.

```
#Nombre archivo: tail.cmd
#Utilización: tail archivo número de líneas.
#Ejemplo: tail prueba.txt 3
@echo off
setlocal ENABLEEXTENSIONS
if {%2} == {} (set "lines=%1") else set "lines=%2"
for /f %a in ('find /c /v "" ^< %1') do set /a skip=%a-lines
for /f "delims=" %a in ('more /e +%skip% ^< %1') do (
echo %a
)
```

4.2.- Módulos de Powershell.

Los módulos son paquetes de comandos de Powershell, que funcionan de manera similar a librerías en otros lenguajes de programación. Por un lado, es posible crear nuestros propios módulos, y por otro, utilizar módulos de comandos desarrollados por terceros para utilizarlos de forma muy sencilla. Una vez importemos un módulo en nuestra sesión de Powershell, podremos utilizar los comandos contenidos en él sin problemas.

Los módulos hacen falta, por ejemplo, para trabajar con Directorio Activo directamente con Powershell. Los comandos asociados a AD en Powershell se encuentran almacenados en el módulo "Active Directory", y es posible que haya que instalarlo para poder hacer uso de ellos.

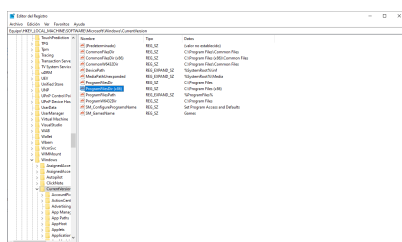
El cmdlet `Get-Module` nos muestra los módulos que tenemos instalados, y con `Import-Module` podemos importar los que necesitemos.

Debes conocer

En este enlace, podemos aprender a trabajar con los módulos de Powershell: importarlos, quitarlos, buscar los disponibles...

[Introducción a los módulos de PowerShell](#)

4.3.- Acceso al registro.



Windows 10 (Elaboración propia)

Primero de todo, hay que comentar que sólo el sistema operativo Windows, en sus distintas versiones, mantiene una base de datos que almacena las configuraciones y opciones en sus versiones de 32 bits, 64 bits y Windows Mobile.

El registro de Windows mantiene una información y configuración de todo el hardware, software, usuarios y preferencias del equipo. Esta información se mantiene jerarquizada.

Para saber más

El siguiente enlace de Microsoft ahonda en los detalles del registro. Cómo es su estructura, qué objetos gestiona, configuración de los distintos usuarios, etc.

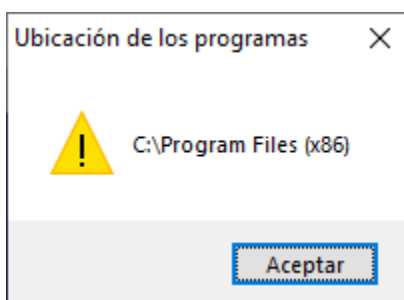
[Información sobre el registro de Windows.](#)

En este otro enlace, vemos cómo trabajar con el registro de Windows desde Powershell

[Trabajar con las entradas del registro desde Powershell](#)

Sabemos que accedemos al registro mediante las órdenes `regedit.exe` o `regedt32.exe`. ¿Pero podemos acceder a dicho registro mediante un fichero de script? Sí, podemos.

¿Cómo podemos acceder al registro? Pondremos un ejemplo para que se entienda bien. Queremos conocer la ruta de acceso a los programas mediante un script escrito en Powershell:



Windows 10 (Elaboración propia)

```
# Creamos una variable y almacenamos en ella la entrada correspondiente del registro
$Entrada = (New-Object -ComObject WScript.Shell).RegRead("HKLM\SOFTWARE\Microsoft\Windows\Cur
# Mostramos un cuadro de diálogo de Windows con los datos leídos del registro
<br>[System.Windows.MessageBox]::Show($Entrada, 'Ubicación de los programas', 'OK', 'Warning')
```

Autoevaluación

**GNU/Linux utiliza, como motor de base de datos de registro, Mysql.
¿Verdadero o falso?**

- ☐ Verdadero.
- ☐ Falso.

No es correcto. Creo que no has leído bien la unidad.

Muy bien, ¿has visto el enlace aconsejado?

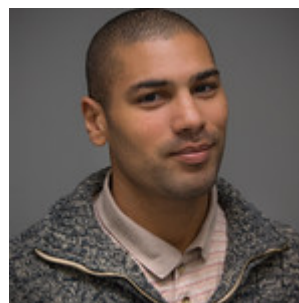
Solución

1. Incorrecto
2. Opción correcta

5.- Tareas administrativas.

Caso práctico

Siguiendo con el enfoque práctico por parte de **Vindio** y **Laro**, quieren plantear situaciones en las que puede ser interesante tener un pequeño programa hecho en script para realizar alguna tarea sencilla y cotidiana que permita ver el alcance y potencia que pueden llegar a tener este tipo de programas de scripts.



[Alain Bachellier](#) (CC BY-NC-SA)

¿Qué entendemos por tareas administrativas? Son aquellas que se realizan de forma repetitiva o rutinaria pero, podríamos llamarlo así, obligatorias. Un ejemplo, realizar copias de seguridad de un dispositivo de almacenamiento masivo a otro. Podemos hacerlo de forma manual, escribiendo los comandos con sus parámetros correspondientes o bien realizamos un script o guión que realice esas tareas de forma automática. Si a esto añadimos una tarea programada, prácticamente nos despreocupamos de realizar la tarea, sólo nos queda comprobar si se ha realizado correctamente.

En cualquier sistema informático se deben realizar, con cierta periodicidad, determinadas acciones o tareas (algunas de ellas, internas del sistema operativo; otras definidas por el administrador; e incluso algunas, definidas por un usuario, con los privilegios adecuados). Por lo tanto, todas aquellas tareas que se ejecuten de forma periódica se denominan tareas programadas.

La programación de tareas permite programar la ejecución de un determinado programa en un momento concreto. Por ejemplo, se puede programar una copia de seguridad, enviar un fichero, comprobar la seguridad del sistema, enviar un informe, etc.

Algunas de las ventajas que se obtienen de estas tareas programadas son las siguientes:

- ✓ Automatización de la gestión del sistema con la finalidad de que se lleven a cabo a pesar de posibles olvidos o descuidos por parte del administrador
- ✓ Se ejecutan en un momento preciso (día y hora)
- ✓ Ayudan o detectan situaciones de error
- ✓ Facilitan el control del sistema
- ✓ Posibilidad de creación de tareas personalizadas por cada usuario

Existen muchas funcionalidades que se pueden llevar a cabo mediante tareas programadas, entre las que se encuentran las siguientes:

- ✓ Generación de informes periódicos (fin de mes, semanal, etc)
- ✓ Comprobación del estado de las comunicaciones.
- ✓ Borrado de ficheros temporales (/tmp, /var/tmp)
- ✓ Tareas de respaldo de información.
- ✓ Control de procesos presentes en el sistema
- ✓ Parada del sistema según horarios de trabajo.
- ✓ Recordatorios
- ✓ Descarga de software en horarios de poco tráfico.

Pongamos un ejemplo de tarea administrativa asociada al arranque del sistema. Los usuarios necesitan varias unidades lógicas y, a través del perfil del usuario, sólo permite una unidad. Entonces necesitamos un script que realice esta tarea. Utilizaremos un script .ps1 y lo colocaremos en la carpeta C:\Windows\SYSVOL\domain\scripts, para que se ejecute en el arranque del sistema.

```
# Sincronizamos la hora con el servidor
Invoke-Command -ComputerName "CORE" -ScriptBlock {Get-Date} | Set-Date -Date
# Montamos la unidad lógica necesaria en K:
New-PSDrive -Name "K" -PSProvider FileSystem -Root "\\CORE\datos-red" -Persist
```

Para hacer algo parecido en GNU/Linux (tareas administrativas en el inicio del sistema), existe un script inicial donde se les asignan, por defecto, varios valores a todos los usuarios, /etc/profile. Aparte de este archivo de inicio, tenemos, en el área de trabajo del usuario, un script de inicio llamado .bashrc que se ejecutará cuando se acceda a través de terminal. Y otro de salida, .bash_logout que se lanzará cuando el usuario dé por terminada la sesión. Editando esos ficheros, podemos añadir tareas a nuestro arranque sin problema.

```
#!/bin/bash
mount -t smbfs -o username=usuario1 //192.168.0.168/datos-red /datos-red
```

Este script, que ha de ser invocado por el superusuario, es capaz de montar una unidad remota de Samba en una carpeta local, a nombre del usuario indicado. Nos pedirá la contraseña, pero también se la podemos pasar a través de un archivo oculto con las credenciales, y pasarle el nombre de usuario por argumento en lugar de que sea siempre "usuario1". Es una versión muy simple, y admite muchas mejoras.

Una de las grandes ventajas del uso de scripts, es que no hay que realizar ningún tipo de adaptación de nuestro sistema para poder ejecutarlos. Eso sí, los permisos y privilegios del programa deben ser adecuados a la tarea que deben realizar. ¿Qué queremos decir? Pongamos un ejemplo que lo clarifique. Si una tarea la debe realizar el supervisor del equipo, el sistema asume la autoría como del supervisor independientemente de quién sea el usuario propietario del programa en sí. Ahora bien, si el programa tiene como propietario al supervisor, ningún usuario deberá poder ejecutarlo. Por ejemplo, un apagado planificado, o una actualización, son tareas que sólo puede ejecutar el administrador del sistema, y no cualquier usuario, y eso tendremos que tenerlo en cuenta.

En algunos scripts, será necesario comprobar si el usuario que los está utilizando es el administrador o no, antes de poner en marcha las tareas; por ejemplo, una actualización del sistema, un apagado o reinicio del servidor..

Veamos cómo se haría en cada sistema:

[En Windows Powershell](#)

[En GNU/Linux Bash](#)

En Windows Powershell

Powershell nos permite hacer esto de una forma muy sencilla. Basta con incluir la siguiente línea al principio del script, y ella se encarga de salir y dar un mensaje descriptivo si el usuario que invoca el script no es el administrador:

```
#Requires -RunAsAdministrator
```

También podemos usar un método más manual, y crear un script expresamente para ello.

```
# Código reutilizable
function Test-Administrator
{
    [OutputType([bool])]
    param()
    process {
        [Security.Principal.WindowsPrincipal]$user = [Security.Principal.WindowsIdentity]::GetCurrent()
        return $user.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator);
    }
}

if(-not (Test-Administrator))
{
    # Aquí pondremos el código oportuno para mostrar el mensaje de error que estimemos oportuno
    Write-Error "Este script debe ser ejecutado por el usuario Administrador.";
    exit 1;
}

$ErrorActionPreference = "Stop";

# A continuación, el código del script propiamente dicho
```

En GNU/Linux Bash

En Bash, también podemos comprobar fácilmente si el usuario que ha ejecutado el script es root o no, y actuar en consecuencia. Un ejemplo de código válido para ello sería el siguiente:

```
#!/bin/bash
if [[ $(id -u) -ne 0 ]] ; then
    echo "Este script debe ser ejecutado por el usuario root"
    exit 1
fi
```

Debes conocer

En este sitio de Microsoft tienes un buen montón de scripts de Powershell para realizar tareas administrativas de ejemplo; principalmente, gestión de procesos, servicios y equipos.

[Scripts de Powershell para tareas administrativas](#)

En este otro, tenemos scripts para gestión de procesos y señales en GNU/Linux

[Gestión de señales y procesos en Bash](#)

Y en esta otra, múltiples ejemplos de scripts para Bash

[Ejemplos de scripts para Bash](#)

Por último, una 'chuleta' o 'cheatsheet' completa del lenguaje y estructuras de Bash

[Bash scripting cheatsheet](#)

6.- Programación para la gestión de directorios.

Caso práctico



[Alain Bachellier](#) (CC BY-NC-SA)

Laro y **Vindio** quieren afrontar y enfocar, en sus respectivas ponencias, cómo acceder a los directorios activos de los sistemas operativos: en Windows con el Active Directory y en GNU/Linux con LDAP.

Una de las tareas administrativas más importantes para llevar a cabo es la administración y gestión de directorios, bien sea Directorio Activo de Microsoft o OpenLDAP de GNU/Linux, que son los dos que hemos visto en este curso.

Algunas tareas repetitivas de las que se llevan a cabo con esos directorios, como pueden ser instalación de estos servicios, copia de seguridad de los objetos, búsquedas, migraciones, actualizaciones de datos... pueden ser implementadas en scripts, con el consiguiente ahorro de tiempo que ello conlleva.



[davereid20](#) (CC BY-NC-SA)

6.1.- «Scripts» para la administración de Directorio Activo.

En esta ocasión vamos a empezar con un ejemplo realizado en Windows Server y con un script realizado en Powershell. Este script consiste en extraer los usuarios de un grupo, "creativos", que está dentro de la unidad organizativa "remotos" y en el dominio "bk.com".

```
# Almacenamos la búsqueda en un array
$usuarios = Get-ADUser -Filter * -Properties MemberOf | Where-Object MemberOf -Like "CN=creativ

# Recorremos la lista de usuarios
foreach ( $usuario in $usuarios ) {
Write-Host $usuario
}
```

Otro ejemplo. Este script está realizado en Powershell y genera varios usuarios en la unidad organizativa "remotos".

```
#Empezaremos importando el Módulo de Active Directory
Import-Module ActiveDirectory

#Aquí le teclearemos la Ruta donde se encuentra el Fichero *.CSV que contendrá los datos de l
[String]$Ruta = Read-Host "Trayectoria donde se Localiza el archivo USUARIOS.csv"

#En la siguiente sentencia le indicamos que vamos a usar una Unidad Organizativa llamada "rem
$ou="OU=remotos" + "," + (Get-ADDomain).DistinguishedName

$dominio=(Get-ADDomain).DNSRoot

#Importamos el Fichero CSV y posteriormente creará los Usuarios dentro del Dominio Actual y d
Import-Csv -Path $Ruta | foreach-object {$UPN = $_.CUENTA + "@" + "$dominio"
New-ADUser -SamAccountName $_.CUENTA -UserPrincipalName $UPN -Name $_.NOMBRE -DisplayName $_.
```

Previo a la creación del script, es necesario crear un fichero con los datos de estos nuevos usuarios que vamos a introducir, un fichero .csv. Esta forma de trabajar es muy común cuando trabajamos con scripts. En el siguiente enlace, se explica cómo crear un fichero adecuado para este script, y más datos sobre su uso:

[Script de creación de usuarios AD con Powershell](#)

Para saber más

En el siguiente enlace encontrarás ejemplos de Powershell para trabajar con Directorio Activo sobre máquinas con sistema operativo Windows Server.

[Comandos powershell para gestión de AD](#)

6.2.- «Scripts» para la administración de cuentas de usuario y LDAP en GNU/Linux.

En el caso de GNU/Linux existen diversas formas de crear programas de scripts para tareas administrativas, y esto incluye gestión de cuentas de usuario, o del directorio LDAP.

Veamos un ejemplo de script, que muestra si existe un usuario o grupo, UsuarioGrupo.sh:

```
#!/bin/bash
clear
# Leemos usuario y grupo, también podrían pasarse por argumentos<br>
read -p "Introduce grupo... " group<br># Buscamos el usuario en el
if `grep -e "^$user:.*" /etc/passwd >/dev/null`
then<br># Si existe el usuario, hay que ver si existe el grupo
    if `grep -e "^$group:.*" /etc/group >/dev/null`
    then
        echo "Usuario y grupo ya existen en el sistema"
# Si el usuario no existía, miramos el grupo<br>elif `grep -e "^$gr
then
    echo "usuario no existe, grupo si!!"
else
    echo "Ni grupo ni usuario existen"
fi
```



[David Schoen / lyte](#) (Dominio público)

En este otro script, añade una foto por defecto a los usuarios que no tuvieran una previamente. Lo que hace es generar un fichero .tar, que después podrá ser procesado por LDAP sin problemas.

```
#####
#!/bin/bash

PHOTO=/tmp/tux.jpg

IFS=$'\n'

for dn in $(ldapsearch -ZZ -LLL -A -b
'ou=Users,ou=Intranet,o=Company,c=NL' -s one
'(&(!(jpegPhoto=*)))(objectClass=inetOrgPerson))' jpegPhoto \
| unldif | grep '^dn' )
do
echo $dn
echo "changetype: modify"
echo "add: jpegPhoto"
```

```
echo "jpegPhoto:$(openssl base64 -in $PHOTO | sed 's/^/ /')"  
echo  
done  
#####
```

Debes conocer

En este enlace, encontramos desarrollado y bien documentado un script que sirve para crear usuarios nuevos en LDAP. Podemos encontrar muchos ejemplos de este tipo en Github.

[Script de creación de usuarios LDAP con Bash](#)

[Creado con eXeLearning \(Ventana nueva\)](#)